

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

TRICHY CAMPUS | DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FULL STACK WEB DEVELOPMENT**TASK 2: INTERACTIVE JAVASCRIPT AND REACTJS APPLICATION DEVELOPMENT****Subject Code: 21CSE354T**

STUDENT NAME	Dharshini E
REGISTER NUMBER	RA2411003050048
PROGRAM / SECTION	B.Tech CSE - A
YEAR / SEMESTER	3rd YEAR / 5th SEMESTER
ASSIGNMENT / PROBLEM NO.	Task 2 (Problem No. 22 – Restaurant Table Booking)
TASK 1 NAME	Aurelia Restaurant (Responsive Web Design)
TASK 1 REPO LINK	https://github.com/edharshini826-code/Aurelia-Restaurant
TASK 2 NAME	Aurelia Restaurant (Restaurant Table Booking - ReactJS)
TASK 2 REPO LINK	https://github.com/edharshini826-code/_____
TASK 2 LIVE LINK	https://edharshini826-code.github.io/_____
DATE OF SUBMISSION	01-10-2026
COURSE	Full Stack Web Development (21CSE354T)
FACULTY COORDINATOR	Dr. P. Hariharan, AP, CSE
ACADEMIC YEAR	2026 – 2027

“Design • Develop • Deploy”

RESTAURANT TABLE BOOKING

Smart Interactive Table Reservation & Dining Experience System

1. TITLE

Project Title: Restaurant Table Booking — Interactive ReactJS Table Reservation & Dining Experience System

Problem Statement No. 22: Display tables, accept booking details, validate information, and show confirmation.

Restaurant Table Booking is an interactive, responsive Single-Page Application (SPA) developed using React 18, JSX, and Vite. The platform addresses Problem Statement 22 by implementing an interactive visual floor plan for table selection across distinct dining zones, a controlled reservation form with real-time client-side validation, cross-session state persistence via browser localStorage, nominal pre-booking deposit calculations, and dynamic booking confirmation dashboards.

2. ABSTRACT

Restaurant Table Booking (Aurelia Restaurant) is a modern client-side web application developed to provide a seamless table reservation workflow inside a web browser, eliminating the need for cumbersome full-page reloads. Built using React 18 and Vite, the application fulfills all requirements of **Problem Statement 22 (Restaurant Table Booking)** from the Full Stack Web Development curriculum.

The application features an interactive visual floor map presenting 11 distinct dining tables across five architectural zones: Window View, Main Dining Hall, Terrace Garden, Family Sovereign, and Private VIP Lounges. Tables dynamically indicate availability status (Available, Selected, or Occupied) with automated capacity matching tailored to the guest count.

Upon selecting a table, patrons enter reservation details into a controlled form. The application intercepts form submission and evaluates inputs against strict client-side validation rules: verifying non-empty alphabetical guest names (≥ 3 characters), standard RFC email patterns, strict 10-digit mobile phone numbers, non-past dates, time slot selection, and mandatory table selection. If any constraint fails, targeted visual error messages appear inline with red border field highlights. Upon successful validation, the reservation is confirmed, a nominal ₹100 pre-booking confirmation token is credited, and details are synchronized to browser localStorage. The project also provides integrated food pre-ordering and guest waiting lounge games (Memory Match and 8-Tile Puzzle), demonstrating competencies in component modularity, React hooks (useState, useEffect, useContext, useMemo), and immutable state management.

3. INTRODUCTION

Table reservation interfaces are a fundamental component of hospitality engineering, serving as the primary digital gateway for guests seeking guaranteed seating and personalized dining arrangements. Traditional server-rendered or multi-page restaurant websites often suffer from full-page reloads, loss of reservation progress during navigation, and lack of real-time input validation, leading to user frustration and double bookings.

The Restaurant Table Booking application was engineered as part of Task 2 (Interactive JavaScript and ReactJS Application Development) under the Full Stack Web Development course (21CSE354T), building upon the responsive styling foundation of Task 1 (<https://github.com/edharshini826-code/Aurelia->

Restaurant). Rather than relying on static HTML forms or external booking plugins, the system is architected as an autonomous Single-Page Application (SPA) driven by React functional components.

The interface couples an interactive dining floor map with controlled form validation, dynamic state transitions, and persistent storage. Through centralized state management via React Context, guests can seamlessly transition between table reservations, culinary pre-ordering, dynamic billing calculations, and waiting lounge entertainment without a single page reload.

4. PROJECT CONCEPT / USE CASE

4.1 Project Concept

Restaurant Table Booking models a complete digital hospitality management system operating directly within the web browser. The execution workflow follows a clear, uninterrupted user journey:

- **Floor Map Exploration:** The guest views available dining tables mapped across 5 ambiance zones (Window View, Main Hall, Terrace Garden, Family Sovereign, VIP Suite).
- **Table Selection & Fit Matching:** Clicking an available table marks it as selected, highlighting recommended tables based on party size.
- **Controlled Reservation Entry:** The user enters guest name, email, 10-digit mobile phone, reservation date, dining time slot, and special culinary requests.
- **Client-Side Validation:** JavaScript validates input strings against regex constraints, displaying inline error banners and highlighting invalid fields if rules fail.
- **Reservation Confirmation & Persistence:** Upon valid submission, the application updates global state, saves data to localStorage, and renders an active reservation dashboard with ₹100 deposit credit.
- **Linked Food Pre-Ordering:** Patrons optionally curate multi-course meals from the 20+ item menu, dynamically calculating subtotals, GST (5%), and deposit token deductions.
- **Guest Lounge Mini-Games:** Guests awaiting dining arrival play interactive memory match and 8-tile sliding puzzle games.

4.2 Target Users

- **Dining Patrons & Event Hosts:** Guests seeking effortless table booking with zero waiting lines and personalized seating arrangements.
- **Hospitality Managers & Front-Desk Staff:** Restaurant staff monitoring real-time seating availability and customer arrival schedules.
- **Web Development Students:** Learners studying controlled forms, client-side validation, React hooks, Context API, and localStorage persistence.

5. OBJECTIVES

- **Problem 22 Fulfillment:** To display tables, accept booking details, validate information, and show confirmation within a responsive React application.
- **Declarative UI Construction:** To build an accessible Single-Page Application using React 18, JSX, and semantic HTML5 elements.
- **Multi-View SPA Routing:** To engineer client-side routing across six functional views (Home, Booking, Menu, Cart, Lounge, Reviews) without browser refreshes.
- **Interactive Floor Plan Map:** To develop an interactive table layout component showing 11 tables across 5 zones with real-time available/occupied statuses.

- **Client-Side Form Validation:** To implement strict validation rules for name (≥ 3 letters), email (RFC format), phone (10 digits), future dates, and table selection.
- **Centralized State Management:** To establish a global state architecture using React Context API (CartContext) for reservations, cart items, and toast alerts.
- **LocalStorage Persistence:** To synchronize reservation and cart data with browser localStorage for uninterrupted session recovery upon reload.
- **Dynamic Financial Computations:** To compute pre-booking deposit credits (₹100 token), GST (5%), promotional discounts (AURELIA15), and grand totals.
- **Interactive Guest Entertainment:** To build stateful mini-games (Memory Match and 8-Tile Sliding Puzzle) for customer entertainment.
- **Bloom's Taxonomy Mastery:** To demonstrate K3 (Apply) through hook implementations and K4 (Analyze) through modular component separation.

6. TECHNOLOGIES USED

6.1 HTML5 & Semantic Elements

Semantic HTML5 markup (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<form>`, `<footer>`) providing accessible structure and responsive viewport scaling.

6.2 CSS3 & Responsive Design Tokens

Luxury obsidian and gold styling implemented using CSS Custom Properties (`variables.css`), CSS Grid for floor plan tables, and Flexbox for forms and toolbars.

6.3 JavaScript ES6+

Modern ECMAScript features including arrow functions, object/array destructuring, template literals, spread syntax, and regex validation expressions.

6.4 ReactJS 18 & JSX

Virtual DOM reconciliation, modular functional components, declarative JSX templates, and unidirectional state flows.

6.5 React Hooks & Context API

`useState` for local component state, `useEffect` for localStorage synchronization, `useContext` for global cart/booking access, and `useMemo` for performance-optimized filtering.

6.6 Web Storage API (localStorage)

Native browser Web Storage API used to persist active table reservations and cart items as serialized JSON across browser sessions.

7. FEATURES & FUNCTIONALITIES

- **7.1 Multi-View SPA Navigation:** Instant view switching across Home, Table Booking, Menu, Cart & Bill, Guest Lounge, and Reviews with automatic scroll restoration.
- **7.2 Interactive Floor Map (TableMap.jsx):** Visual layout presenting 11 tables across 5 zones (Window View, Main Hall, Terrace Garden, Family Sovereign, VIP Suite) with live availability.
- **7.3 Controlled Reservation Form (Booking.jsx):** Controlled React form handling guest name, email, 10-digit phone, reservation date, dining slot, party size, and special dining notes.
- **7.4 Real-Time Client-Side Validation:** Form validation enforcing non-empty names (≥ 3 letters), RFC email regex, exact 10-digit phone numbers, non-past dates, and table selection.

- **7.5 Confirmed Reservation Dashboard:** Renders confirmed guest details, table name, date, party size, ₹100 deposit token credit, and reservation cancellation controls.
- **7.6 Synchronized Food Pre-Ordering:** Integrated 20+ dish menu with instant search, category tabs (Combos, Starters, Mains, Desserts, Drinks), dietary toggles, and price sliders.
- **7.7 Dynamic Cart & Billing Engine:** Line item steppers, coupon vouchers (AURELIA15, LUXURY20), 5% GST computation, and simulated checkout modal with UPI QR code.
- **7.8 Guest Waiting Lounge & Mini-Games:** 12-card culinary memory match game and royal 8-tile sliding puzzle with move counters and victory detection.

8. SYSTEM DESIGN

8.1 Overall Workflow & Execution Flow

- **1. Application Mount:** Vite initializes React root at main.jsx; CartContext hydrates active booking and cart data from localStorage.
- **2. Navigation Selection:** Guest clicks 'Table Booking' on Navbar; view transitions to Booking.jsx without page reload.
- **3. Table Map Inspection:** TableMap renders 11 table cards showing capacity, zone, and availability status.
- **4. Table Selection:** Guest clicks an available table (e.g. Table 1 - Window View); tableId updates and highlights in gold.
- **5. Form Entry:** Guest fills out Full Name, Email, Phone Number, Date, Time, Party Size, and Special Requests.
- **6. Submit Interception:** Submit event is intercepted using event.preventDefault(); validate() evaluates all input fields against pre-defined rules.
- **7. Error Branch:** If validation fails, field-specific error messages render in red and invalid input borders highlight.
- **8. Success Branch:** If validation passes, saveBooking() updates Context state, writes to localStorage, and shows a gold confirmation toast.
- **9. Confirmation Dashboard:** Active reservation card displays table number, guest details, and ₹100 deposit credit.
- **10. Optional Pre-Order:** Guest clicks 'Pre-Order Culinary Course', selects dishes from Menu view, and adds them to cart.
- **11. Financial Recalculation:** Cart view recalculates subtotal, applies coupon code, adds 5% GST, and adjusts ₹100 table deposit.
- **12. Checkout & Invoice:** Guest opens simulated checkout modal, selects UPI QR payment, and receives order confirmation receipt.
- **13. Lounge Entertainment:** Guest accesses Lounge view to play memory match or 8-puzzle while awaiting table seating.
- **14. Modification / Cancellation:** Patron can click 'Cancel Reservation' to clear booking from state and localStorage.
- **15. State Persistence:** Reloading browser preserves confirmed reservation and cart state seamlessly.

8.2 System Components & Architecture

The system is organized into three decoupled layers: User Interface Layer (JSX components styled with CSS tokens), Logic & Validation Layer (React hooks, regex validators, and event handlers), and Persistence Layer (browser localStorage).

9. IMPLEMENTATION

9.1 React Component Hierarchy & Client-Side Routing

App.jsx manages top-level view orchestration, wrapping the component tree inside CartProvider and conditionally rendering active views without browser reloads.

```

1 // 1. App.jsx - Component Architecture & Client-Side View Routing
2 export function App() {
3   const [currentView, setView] = useState('booking'); // Problem 22: Table Booking
4
5   useEffect(() => {
6     window.scrollTo({ top: 0, behavior: 'smooth' });
7   }, [currentView]);
8
9   return (
10    <CartProvider>
11      <div className="app-container">
12        <Navbar currentView={currentView} setView={setView} />
13        <main className="main-content">
14          {currentView === 'home' && <Home setView={setView} />}
15          {currentView === 'booking' && <Booking setView={setView} />}
16          {currentView === 'menu' && <Menu setView={setView} />}
17          {currentView === 'cart' && <Cart setView={setView} />}
18          {currentView === 'lounge' && <Lounge setView={setView} />}
19          {currentView === 'reviews' && <Reviews setView={setView} />}
20        </main>
21        <Footer setView={setView} />
22        <Toast />
23      </div>
24    </CartProvider>
25  );
26 }

```

Figure 1: Component Hierarchy and Client-Side SPA Routing Architecture (App.jsx)

9.2 Interactive Floor Map & Table Selection Logic

TableMap.jsx renders the visual restaurant layout across 5 dining zones, providing dynamic capacity matching and availability indicators.

```

1 // 2. TableMap.jsx - Interactive Restaurant Floor Map & Zone Selection
2 export const TableMap = ({ selectedTableId, onSelectTable, guestCount = 2 }) => {
3   return (
4     <div className="table-map-container luxury-card">
5       <div className="map-header">
6         <h4>Interactive Dining Floor Map (5 Zones)</h4>
7         <div className="map-legend">
8           <span className="dot available">Available</span>
9           <span className="dot selected">Selected</span>
10          <span className="dot occupied">Reserved</span>
11        </div>
12      </div>
13      <div className="tables-grid">
14        {RESTAURANT_TABLES.map((t) => {
15          const isSelected = selectedTableId === t.id;
16          const isOccupied = t.status === 'occupied';
17          const isGoodFit = t.capacity >= guestCount && t.capacity <= guestCount + 2;
18
19          return (
20            <div>
21              <div>
22                key={t.id}
23                className={`table-node ${isSelected ? 'selected' : ''} ${isOccupied ? 'occupied' : 'available'}`}
24                onClick={() => !isOccupied && onSelectTable(t)}
25              </div>
26              <div>
27                <span className="table-name">{t.name}</span>
28                <span className="table-cap">{t.capacity} Guests</span>
29                <span className="table-zone">{t.zone}</span>
30                <span className="status-badge">{isOccupied ? 'Occupied' : isSelected ? 'Selected' : 'Available'}</span>
31              </div>
32            </div>
33          );
34        })}
35      </div>
36    </div>
37  );
38 }

```

Figure 2: Interactive Restaurant Floor Map & Zone Selection Component (TableMap.jsx)

9.3 Controlled Form State & Client-Side Validation Rules

Booking.jsx implements controlled form state with client-side validation rules for names, emails, phone numbers, future dates, and table selection.

```

1 // 3. Booking.jsx - Controlled Form State & Client-Side Validation Rules
2 const validate = () => {
3   const newErrors = {};
4
5   // Rule 1: Full Name constraint (required, >= 3 alphabetic characters)
6   if (!formData.guestName.trim()) {
7     newErrors.guestName = 'Full Name is required.';
8   } else if (formData.guestName.trim().length < 3) {
9     newErrors.guestName = 'Name must contain at least 3 characters.';
10  } else if (!/^[a-zA-Z\s]+$/.test(formData.guestName.trim())) {
11    newErrors.guestName = 'Name should contain only alphabetical characters.';
12  }
13
14  // Rule 2: Email format validation (standard RFC email pattern)
15  if (!formData.email.trim()) {
16    newErrors.email = 'Email address is required.';
17  } else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email.trim())) {
18    newErrors.email = 'Please provide a valid email format (name@domain.com).';
19  }
20
21  // Rule 3: 10-Digit Mobile Number validation
22  if (!formData.phone.trim()) {
23    newErrors.phone = 'Phone number is required.';
24  } else if (!/^\d{10}$/.test(formData.phone.trim().replace(/\s/g, ''))) {
25    newErrors.phone = 'Enter a valid 10-digit mobile number.';
26  }
27
28  // Rule 4: Future reservation date verification
29  const today = new Date().toISOString().split('T')[0];
30  if (!formData.date) {
31    newErrors.date = 'Reservation date is required.';
32  } else if (formData.date < today) {
33    newErrors.date = 'Reservation date cannot be in the past.';
34  }
35
36  // Rule 5: Dining Table selection enforcement
37  if (!formData.tableId) {
38    newErrors.tableId = 'Please select a dining table from the floor map.';
39  }
40
41  setErrors(newErrors);
42  return Object.keys(newErrors).length === 0;
43 };

```

Figure 3: Controlled Form State & Client-Side Validation Logic (Booking.jsx)

10. USER INTERFACE / SCREENSHOTS OF ACTUAL WEBSITE

10.1 Screenshot 1 – Task 1 Live Website Baseline (GitHub Pages)

The foundational Task 1 responsive website (Aurelia Restaurant) deployed live on GitHub Pages at <https://edharshini826-code.github.io/Aurelia-Restaurant/>, featuring warm obsidian-gold luxury branding, hero culinary showcase, and aesthetic typography.

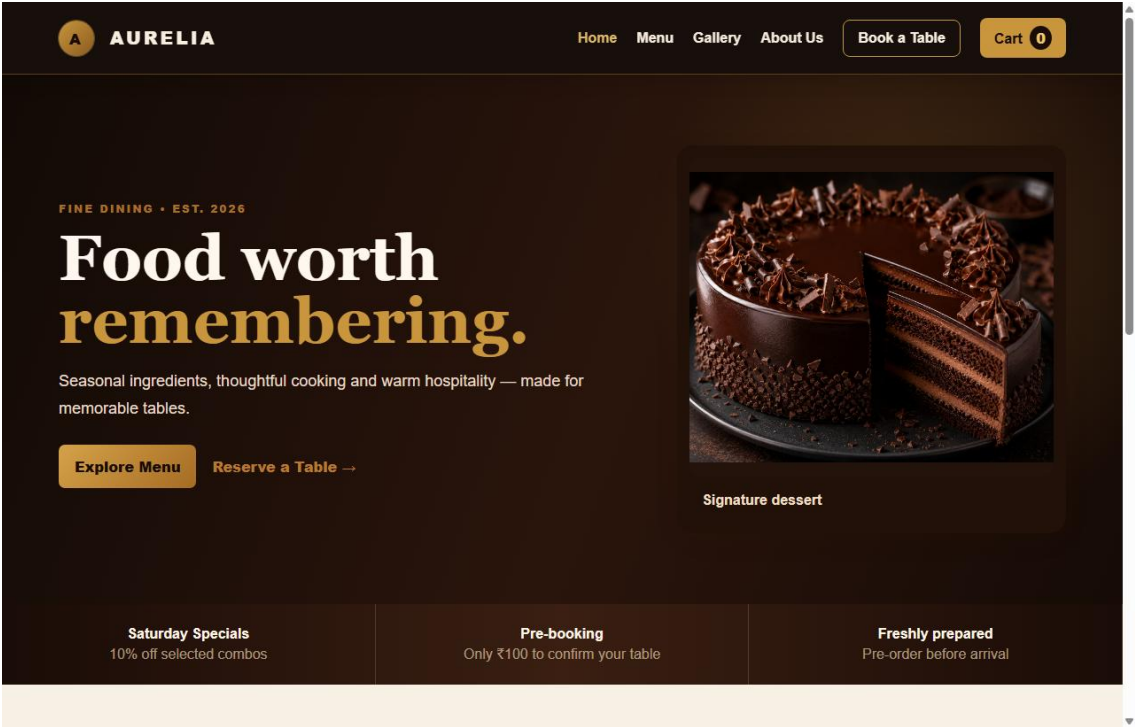


Figure 4: Task 1 live website deployed on GitHub Pages showing luxury branding and culinary showcase

10.2 Screenshot 2 – Task 2 Home & Dining Ambiance Showcase (ReactJS)

The Task 2 ReactJS Single-Page Application Home view featuring dynamic navigation bar, culinary opulence hero banner, key dining metrics (50+ Dishes, 4.9★ Reviews, 100% Organic, VIP Suites), and quick reservation CTA buttons.

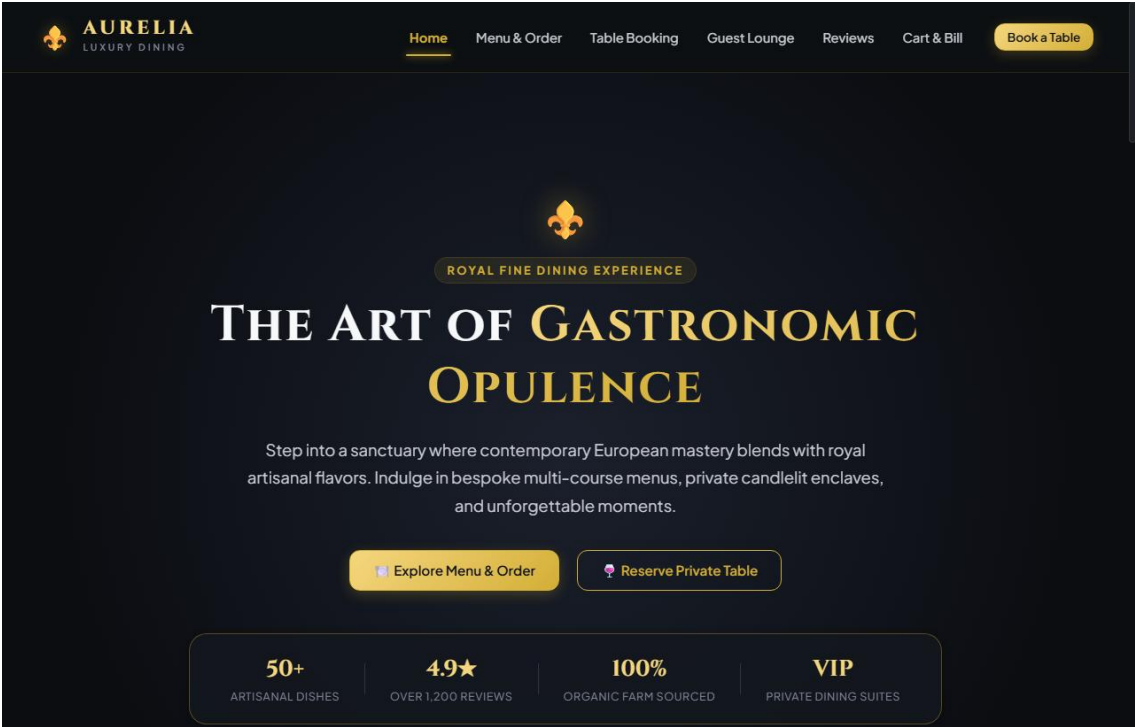


Figure 5: Task 2 interactive Home view featuring SPA navigation and royal dining metrics

10.3 Screenshot 3 – Interactive Table Floor Map Selector (Problem No. 22)

The interactive table reservation view showing the 11-table floor map across 5 ambiance zones (Window View, Main Hall, Terrace Garden, Family Sovereign, VIP Suite) with party-size crown recommendations and controlled guest input fields.

AURELIA
LUXURY DINING

Home Menu & Order **Table Booking** Guest Lounge Reviews Cart & Bill [Book a Table](#)

ROYAL RESERVATIONS

RESERVE YOUR PRIVATE TABLE

Experience our legendary hospitality. Select your seating zone, specify your party details, and complete table pre-booking.

INTERACTIVE DINING FLOOR MAP

● Available ● Selected ● Reserved

Select your preferred table setting. Tables highlighted with a gold crown are optimal for your party size of 2 guests.

Table 1 2 Guests WINDOW VIEW Selected	Table 2 2 Guests WINDOW VIEW Available	Table 3 4 Guests MAIN DINING HALL Available
Table 4	Table 5	Table 6

GUEST DETAILS & PREFERENCES

Fields marked with * are mandatory.

Chosen Table:
Table 1 (Window View)

Full Name *

Email Address *

Phone Number (10 digits) *

Figure 6: Interactive dining floor map with zone availability indicators and controlled reservation inputs

10.4 Screenshot 4 – Confirmed Table Reservation Dashboard & Deposit Credit

Upon successful reservation submission, the application renders the confirmed reservation dashboard for Dharshini E (Table 1 - Window View, 2 Guests, ₹100 pre-booking deposit credit) with state persistence and real-time cancellation controls.

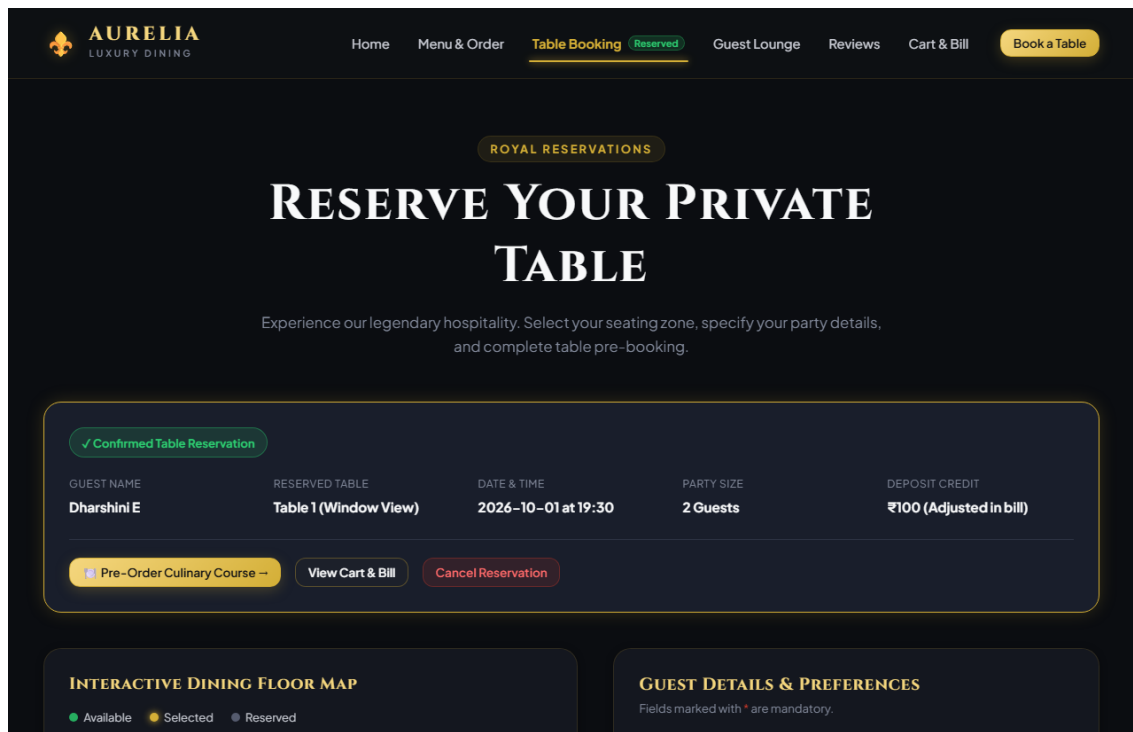


Figure 7: Confirmed Table Reservation Dashboard with state persistence and ₹100 pre-booking deposit credit

10.5 Screenshot 5 – Linked Culinary Menu & Dynamic Billing Engine

The dynamic cart and billing interface displaying selected gourmet courses, automated line-item quantity steppers, 5% GST hospitality tax calculation, promotional voucher engine (AURELIA15), and deposit deduction.

AURELIA
LUXURY DINING

HomeMenu & OrderTable BookingGuest LoungeReviewsCart & Bill 5Book a Table

ORDER & BILLING

YOUR CULINARY
ORDER & BILL

Review your selected courses, apply promotional vouchers, and proceed to confirmed checkout.

5 Delicacies in Order

Clear All Items



GARLIC BUTTER JUMBO PRAWNS

₹680 each

- 2 +

₹1360

×



TRUFFLE MUSHROOM RISOTTO

₹740 each

- 1 +

₹740

×

PROMOTIONAL VOUCHER

TRY AURELIA15 OR LUXURY20

Apply

Tip: Use code AURELIA15 for 15% discount or LUXURY20 for 20% off.

BILLING SUMMARY

Food Order Subtotal:₹2860

GST (5% Hospitality Tax):₹143

Figure 8: Integrated order cart, automated line-item math, 5% GST tax calculation, and billing summary

11. OBSERVED RESULTS

The developed Restaurant Table Booking ReactJS application was evaluated across 19 distinct functional scenarios to verify table selection, form validation rules, financial precision, and cross-session persistence:

Test / Operation	Observed Result
Launch application & rehydrate state	Application loads instantly in browser without errors; CartContext retrieves persisted reservation and cart from localStorage.
Navigate to Table Booking view (SPA)	View transitions instantaneously to Table Booking without full-page reload; floor map renders with 11 tables.
Interactive floor map table selection	Clicking an available table (e.g. Table 1 - Window View) highlights table card in gold and updates form state.
Occupied table click prevention	Clicking an occupied table (e.g. Table 4) is blocked; cursor indicates not-allowed; state remains unchanged.
Party size fit recommendation	Changing guest count to 6 dynamically displays royal crown badge (👑) on 6-seater terrace tables.
Submit completely blank booking form	Displays inline validation errors for Name, Email, Phone, and Table; submission is halted via early return.
Submit name with numbers or < 3 chars	Displays 'Name must contain at least 3 alphabetical characters' error; highlights field with red border.
Submit invalid email format	Entering 'dharshini@' prompts 'Please provide a valid email format (name@domain.com)'; highlights field.
Submit phone with non-digits or != 10 chars	Entering '98765' prompts 'Enter a valid 10-digit mobile number'; highlights phone input.
Submit past reservation date	Selecting yesterday's date prompts 'Reservation date cannot be in the past'; submission blocked.
Valid reservation form submission	Validation passes 100%; saves reservation to Context; shows gold toast alert; displays active reservation card.
Pre-booking deposit token calculation	Nominal ₹100 pre-booking confirmation token is automatically recorded and credited toward the dining bill.
Cancel active reservation	Clicking 'Cancel Reservation' removes booking from Context and localStorage; clears active card immediately.
Integrated food pre-ordering	Navigating to Menu view allows guest to select courses; items sync with table reservation seamlessly.
Apply promotional coupon (AURELIA15)	Entering 'AURELIA15' applies 15% discount to pre-ordered food; recalculates subtotal, 5% GST, and total.
Simulated checkout modal & UPI QR	Clicking 'Proceed to Checkout' opens modal; selecting UPI displays payment QR code; submitting generates order invoice.

Test / Operation	Observed Result
Culinary memory match mini-game	Clicking cards flips them; matching pairs remain revealed with green borders; move counter tracks score accurately.
Royal 8-tile sliding puzzle	Clicking tiles adjacent to empty slot moves tile into space; move counter increments; victory check detects solved state.
Page refresh / cross-session persistence	Reloading browser preserves confirmed table reservation and cart items via localStorage persistence.

12. LIMITATIONS

- **Client-Side Persistence Scope:** Reservation details persist locally in browser localStorage, meaning reservations are scoped to the individual client machine.
- **Mock Payment Gateway:** The pre-booking confirmation token simulates UPI QR payments without connecting to a live commercial payment gateway API.
- **Static Table Inventory:** Table configurations and initial availability statuses are served from a local data module rather than synchronized via live database queries.
- **Single-Terminal Floor Plan:** Table concurrency across multiple simultaneous users over WebSockets is not implemented in this front-end phase.

13. FUTURE ENHANCEMENTS (TASKS 3 & 4 ROADMAP)

- **Task 3 (Spring Boot REST API & Database):** Migrate table inventory and booking records to a Spring Boot backend with Spring Data JPA and MySQL/PostgreSQL, providing REST endpoints (POST /api/v1/bookings, GET /api/v1/tables).
- **Task 4 (Cloud Microservices):** Decompose system into dedicated microservices (Booking Service, Catalog Service, Order Service, Notification Service) managed via Spring Cloud Gateway and Docker.
- **Live Payment Gateway:** Integrate Razorpay or Stripe Webhook SDKs for real-time UPI token verification and digital invoice generation.
- **Multi-User WebSocket Synchronization:** Implement WebSocket channels to lock tables in real time when selected by other patrons across devices.

14. CONCLUSION

The Restaurant Table Booking (Aurelia Restaurant) project successfully fulfills all requirements of Problem Statement 22 by demonstrating the development of an interactive, responsive Single-Page Application using React 18, JSX, and Vite. By integrating a visual table floor plan, client-side regex form validation, global Context state management, and persistent storage via the Web Storage API (localStorage), the application provides a smooth, reliable reservation workflow without full-page reloads.

Through this project, key competencies across Bloom's Taxonomy levels K3 (Apply) and K4 (Analyze) were rigorously demonstrated. Front-end engineering concepts—including event interception, controlled input handling, conditional validation logic, dynamic DOM updates, and cross-session persistence—were thoroughly mastered. The resulting modular architecture provides an exceptionally solid foundation for upcoming full-stack database integrations in Tasks 3 and 4.

15. REFERENCES

- **React Official Documentation:** react.dev – React Functional Components, JSX Syntax, Hooks (useState, useEffect, useContext, useMemo), and Context API.
- **Vite Official Documentation:** vitejs.dev – Next Generation Frontend Tooling, Hot Module Replacement (HMR), and Rollup Production Bundling.
- **MDN Web Docs:** developer.mozilla.org – Web Storage API, localStorage Persistence, and JSON Serialization.
- **MDN Web Docs:** developer.mozilla.org – HTML Forms, Input Validation, and Form Events.
- **MDN Web Docs:** developer.mozilla.org – CSS Grid Layout, Flexible Box Layout, and CSS Custom Properties.
- **SRM Institute of Science and Technology:** 21CSE354T Full Stack Web Development Curriculum and Task 2 Assignment Guidelines (Problem Statement 22).